When the Loop Is Inert: Constructible Attractors, Search, and Tool Use in LLM-Guided Circle Packing

An Auditable Four-Arm Study with Zero-Shot Controls and an Open-Weight Precision Sweep

Authors: Soham Gugale **Affiliation:** South Forsyth High School, Cumming GA **Target Venues:** ISEF 2027, GECCO 2027 (poster / late-breaking) **Date:** July 2026

Abstract

LLM-guided evolutionary search puts an LLM proposer inside a selection loop. Prior work reports single runs on benchmarks whose best known solutions are publicly catalogued, leaving open whether the loop *searches* or merely reproduces what the model can already produce. We answer with a fully auditable study of circle packing (N=26, maximize the sum of radii): four arms of 50 candidates each (5 seeds × 10 generations), 100-sample zero-shot and direct-knowledge controls, tool-augmented probes, and a two-scale open-weight precision sweep — 834 proposer invocations (the 14B rung additionally re-executed bit-identically with coordinate logging), every candidate logged live at evaluation time and scored by a deterministic evaluator.

Three findings. **First, the loop — tested as single-lineage elitist hill-climbing at 50 evaluations per arm, the small-budget regime — is inert on this benchmark because of a constructible attractor, not retrieval.** Two Claude tiers converge to the same 2.5414 grid-and-filler packing — a configuration in no catalog, trivially constructible from first principles — while the catalogued near-optima (≈2.634 since 2012; best known 2.6359) are never emitted, and 6/6 direct knowledge probes disclaim knowing any published value for this objective. The decisive control: best-of-50 independent zero-shot samples reaches *exactly* the loop's converged value in both tiers at matched invocation budget (0/95 valid samples exceed it) — the loop's contribution is selection over the proposer's output distribution, nothing more. Cross-seed variance returns only on a rectangle variant with no natural template. **Second, artifact and tools move the ceiling; looping does not.** Evolving *programs* instead of coordinates restores genuine search (5/5 seeds improve mid-run) yet stays below the attractor at matched budget while viability collapses to 66%; unconstrained tool-using agents exceed the attractor in a single invocation (all five probes beat it; best 2.63598, matching the contemporary record — a single-observation comparison). **Third, weight precision is second-order to scale — until novelty.** A Qwen2.5-Coder sweep (7B: FP16→Q2_K; 14B: Q8_0→Q2_K) shows no viability cliff anywhere from 16 to 3.35 effective bits; the 7B model sits below a capability floor where a *format lottery* makes 2-bit look better than FP16. At 14B, seed-level improvement collapses from 15/15 to 1/5 below 3.91 effective bits ($p = 0.001$) as parent-copying spikes from 14% to 94% of valid outputs — a **novelty cliff**: a bit-identical coordinate-logged re-execution of the run verifies that at Q2_K, 17/18 valid outputs are verbatim coordinate copies of the parent (statistics still rest on one sampling run; fresh-seed

replication is future work). Pass/fail metrics miss both effects. Harness, per-candidate logs, and all probe outputs are retained.

1. Introduction

Evolutionary algorithms solve black-box optimization by iterating variation and selection. Recent systems replace the hand-crafted variation operator with an LLM proposer (FunSearch, AlphaEvolve, OpenEvolve, ShinkaEvolve, CALM; §2), reporting strong results — typically from a single run on the circle-packing benchmark, an objective whose best known values have been publicly catalogued since 2011 (Friedman's *Circles in Squares* tables; the N=26 record stood at ≈2.634 from 2012 until AlphaEvolve's 2.63586 in 2025). Two habits of this young literature motivate our study: single-run reporting, and no accounting of how often the proposer's output is even usable.

We ask five questions. (1) Does the loop *search*, or does it surface an artifact the model can already produce — and if the latter, is that artifact *retrieved* from the published record or *constructed* from a template? (2) Is the achievable ceiling a property of the model tier or of the benchmark? (3) What actually governs the cross-seed variance that single-run reporting hides? (4) Does evolving *programs* — the artifact the motivating systems actually evolve — behave differently from evolving coordinate lists? (5) The project's founding question: as the proposer's weight precision drops (FP16 → Q2_K, 3.35 effective bits), where does the loop break?

We answer the first four with a four-arm controlled study plus three kinds of controls — zero-shot probes, a 100-sample best-of-N control at matched invocation budget, and direct knowledge probes — and the fifth with an open-weight precision sweep (7B and 14B GGUF ladders under the same loop protocol), all under an auditability discipline missing from prior work: every candidate is logged at the moment it is evaluated (no post-hoc reconstruction), a deterministic Python evaluator computes all scores (the LLM never grades itself), and format failure (*viability*) is measured separately from geometric failure (*validity*).

Contributions. 1. **The loop is a selection amplifier over a constructible attractor.** All seeds converge to the same 2.5414 grid-and-filler packing the model can produce zero-shot, and nothing in 50 loop candidates per arm (5 seeds × 10 generations) exceeds it (Arm B; replicating pilot Arm A). Three controls locate the mechanism: best-of-50 independent zero-shot samples reaches exactly the loop's value at matched budget in both tiers (0/95 valid samples above it); 6/6 direct knowledge probes disclaim knowing any published value for this objective; and the catalogued near-optima (≈2.634) are never emitted. The attractor is *constructed* (a 5×5 grid of r=0.1 plus a filler — a template any model that understands a grid can produce), not *retrieved* from the published record. 2. **The ceiling belongs to the artifact, not the model.** Haiku and Sonnet — different capability tiers — share the identical 2.5414 ceiling to six decimal places; at n=50 zero-shot samples per tier the canonical-hit rates are statistically indistinguishable (28/45 vs 29/50 valid samples). One Sonnet seed is trapped *below* the ceiling by elitist retention of its own early hex-lattice choice — the loop can lock in a lineage as well as amplify one. 3. **Memorability, not stochasticity, drives seed variance.** On a rectangle variant with no published optimum, cross-seed variance returns (std 0.065 vs 8×10^{-6}) and genuine mid-run

improvements appear. Cross-seed variance is a *benchmark* property. 4. **Program evolution restores search but not the ceiling — even with a compute subsidy.** When the proposer evolves a *program* instead of a coordinate list (Arm D), genuine hill-climbing returns even on the templated task — all 5 seeds log verified improvements (12 events) and disperse to five distinct non-canonical endpoints, whereas every improving coordinate lineage converges to the one constructed attractor — but at matched evaluation budget (50 evaluations, plus up to 30 s of CPU optimization per evaluation that Arms A–C do not get) the best evolved program (2.5371) never reaches the ceiling (2.5414), and viability drops from 98% to 66% as timeouts and crashes replace format errors. Artifact type selects the failure regime: coordinates construct reliably; programs search unreliably. 5. **Tool access at proposal time, not looping, escapes the attractor.** All five unconstrained agents that chose to write classical optimizers exceeded the 2.5414 ceiling (2.528–2.636); the best, a strictly-verified 2.63598, matches the contemporary published record — offered as an illustrative single-shot observation, not a statistical comparison. The interesting frontier is not LLM-as-mutation-operator but LLM-as-orchestrator-of-solvers. 6. **A precision cliff — in novelty, not viability.** Sweeping an open-weight 7B proposer across five GGUF precisions (FP16 → Q2_K, 3.35 effective bits) under the same loop protocol finds viability statistically flat, zero canonical recall at every precision, and a mildly *higher* viability at Q2_K driven by a measurable format-lottery mechanism (confirmed not to be token-cap truncation): at this scale the capability floor, not bit width, binds first. Scaling the sweep to 14B lifts the format floor (44% vs 12% viability at 8-bit) and reveals a sharp seed-level cliff between 3.91 and 3.35 effective bits in *proposal novelty*: seeds improving past the seeded baseline collapse from 15/15 above the boundary to 1/5 below (Fisher $p = 0.001$, primary test), while valid outputs shift from mostly-mutations (14% echo) to almost-exclusively verbatim parent-copies (94%; per-seed distribution in §5.9). Both results are invisible to pass/fail metrics. The original 14B run's per-candidate files were lost to session expiry; the finding stands on a bit-identical coordinate-logged re-execution (its coordinate-level predictions, the only falsifiable ones, held — §3.4, A.8), with fresh-seed replication as future work.

2. Related Work

2.1 LLM-Guided Evolutionary Search

The lineage begins with FunSearch (Romera-Paredes et al., 2024), which pairs an LLM with an automated evaluator in an island-based population loop — a diversity mechanism designed precisely against the lineage lock-in we document in §5.3/§6.3. FunSearch's cap-set constructions are also the strongest counterexample to any *blanket* claim that these loops only reproduce known artifacts: those objects could not have been retrieved or templated, because they did not exist. Our claims are accordingly scoped to evaluation on benchmarks whose targets the proposer can already produce (§2.3); FunSearch's success on a genuinely open target is exactly what our framework predicts for a task with no pre-existing attractor. AlphaEvolve (Novikov et al., DeepMind, 2025) extends this to an LLM-as-mutation-operator agent and reports state-of-the-art circle packing (2.635862 for N=26). OpenEvolve (Sharma, 2025) is the widely-used open reimplementation; ShinkaEvolve (Lange et al., Sakana AI, 2025) reports strong circle-packing results with far fewer samples; CALM (Huang et al., 2025) co-evolves an LLM with heuristic design. Beyond program synthesis, LLMs serve as variation operators across

evolutionary computation: as a crossover operator (Meyerson et al., 2023), as a learned mutation operator in genetic programming (Lehman et al., 2022), for prompt evolution (Guo et al., 2024), and for quality-diversity-driven architecture search (Nasir et al., 2024). These works vary model size, family, and prompting; to our knowledge, none varies benchmark memorability, none reports per-candidate viability, and none reports multi-seed statistics.

2.2 Quantization Effects on Generation

The quantization literature has established that 4-bit is near-optimal for accuracy per total model bits, with quality degrading sharply below (Dettmers & Zettlemoyer, 2023); that perplexity tracks quantized quality on most static benchmarks while instruction-following degrades disproportionately at low bit-widths (Jin et al., 2024); and that low-bit quantization damages multi-step mathematical reasoning far more than perplexity predicts (Li et al., 2025). All of these measure pass/fail or scalar quality on static benchmarks. To our knowledge, no prior work measures quantization's effect on an LLM *inside an evolutionary loop* — specifically, its capacity to propose a novel mutation of a parent solution — which is where our 14B result locates quantization's first casualty (§5.9). Our novelty cliff is consistent with the reasoning-degradation findings (2-bit sits far below every recommended operating point) yet invisible to their instruments: our 2-bit condition is unimpaired on exactly the pass/fail axes those works report.

2.3 The Gap

Our concern is a geometric cousin of the benchmark-contamination problem: evaluation results are uninterpretable when the model may already contain the answer (Sainz et al., 2023; Golchin & Surdeanu, 2024; survey in Deng et al., 2024). Circle packing adds a twist that contamination detection does not cover — the model need not have *memorized* anything for the benchmark to be compromised, because a near-ceiling solution is *constructible* from a generic template (§5.5). Our best-of-N control also connects to the repeated-sampling literature: Brown et al. (2024) show that coverage under repeated sampling scales smoothly with the number of draws across four orders of magnitude — our control is the same instrument pointed at a different question, using matched-budget repeated sampling not to *improve* performance but to *diagnose* whether a selection loop adds anything beyond it. Results on this benchmark therefore cannot distinguish search from reproduction, whether retrieved or constructed. To our knowledge, no prior work runs the same loop on a matched variant with no natural template, compares model tiers at fixed benchmark, runs a best-of-N control at matched invocation budget, separates format failure from geometry failure per candidate, or compares solution-space against program-space evolution inside one harness. We do all five, with live-logged data.

3. Method

3.1 Tasks

Unit square (documented, template-friendly): pack N=26 non-overlapping circles in $[0,1]^2$ maximizing the sum of radii; best known value 2.63586 (AlphaEvolve, 2025 — not proven optimal),

improving a record catalogued in Friedman's *Circles in Squares* tables since 2012 (≈2.634, Cantrell/Specht). Note the object our models actually converge to — a 5×5 grid of r=0.1 circles plus one filler, sum ≈2.5414 — appears in no catalog we could locate; it is a trivially constructible arrangement, and §5.5 tests retrieval against construction directly. **Rectangle (no documented target, template-hostile):** identical rules in [0,1]×[0,0.83]. To our knowledge no published solution exists for this aspect ratio and N; reproducing a documented configuration cannot supply the answer, though *strategies* (grid + filler) may transfer. This manipulation changes the region, the attainable score scale, and how naturally grid templates fit along with documentation status — a bundled contrast, discussed as a limitation in §6.7.

Score = sum of radii, computed only for valid candidates. **Viable** = output parses to exactly 26 (x, y, r) triples. **Valid** = viable ∧ all circles in-bounds ∧ no pairwise overlap (centre distance ≥ radius sum), with a 10^{-6} tolerance on the boundary and overlap checks (identical in all harness versions). Descriptors: mean radius, packing density.

3.2 Archive and Loop

The loop is **elitist hill-climbing**: per generation, select the parent (the seed's best-so-far) → LLM proposes → deterministic evaluator scores → accept if valid and improving. The harness implements this through a nominal MAP-Elites archive (Mouret & Clune, 2015; 20×20 grid over mean_radius × density), but the descriptors barely vary on this task — coverage never exceeds 6/400 cells (1.5%) in any arm (§5.6) — so the archive is degenerate by construction and we present and analyze the system as plain elitist hill-climbing throughout. QD metrics are reported only for completeness.

3.3 Arms and Controls

Arm	Proposer	Task	Artifact	Constraint
A	Claude Haiku	unit square	coordinate list	agentic, in-practice pure (tool_uses = read+write only)
B	Claude Sonnet	unit square	coordinate list	**no-code rule**: Read/Write tools only, packing from reasoning
C	Claude Haiku	rectangle	coordinate list	no-code rule
D	Claude Haiku	unit square	**Python program**	no-code rule for the proposer; program executed sandboxed by the harness

Arm D protocol (program evolution). The proposer writes a Python program; the harness — not the proposer — executes it and scores its printed output with the same evaluator. Sandbox: imports restricted by AST check to `math`, `random`, `itertools`, `functools` (stdlib-only is deliberate: any numerical refinement must be *evolved into the program*, not imported — this is exactly what separates Arm D from the tool-augmented probes of §5.8, where agents ran scipy at proposal time); banned names (`open`, `exec`, `eval`, …) rejected pre-execution; `python -I` subprocess with a 30 s kill. The champion program's full source is fed back as the parent in the next generation's context. Failures are logged with a taxonomy

(`banned_import`, `banned_name`, `syntax_error`, `timeout`, `crash`, `no_output`, `parse_error`) rather than a single viability bit, because program proposals fail in ways coordinate proposals cannot.

5 seeds (42, 123, 456, 789, 1111) × 10 generations per arm = 200 candidates. Controls, three kinds. *Zero-shot probes* (single query, no loop): 6 Haiku/unit (pure), 6 Sonnet/unit (no-code), 6 Haiku/rectangle (mixed), plus 6 *unconstrained* Sonnet/unit probes in which agents were free to use tools (5 of 6 files produced; agents autonomously wrote SLSQP/basin-hopping optimizers — reported separately as the tool-augmented condition, §5.8). *Best-of-N control* (added in revision, at a reviewer's request): 50 further independent zero-shot samples per tier — the same invocation budget as one loop arm — with a fixed no-tools prompt, each output archived verbatim and scored by the same evaluator; this condition is direct-reply rather than file-writing (a disclosed mode difference from the original probes). *Direct knowledge probes*: 3 per tier, asking for the best known value and source for this exact objective, with an explicit instruction to say plainly if unknown (verbatim responses archived).

Arms B and C use one fixed prompt for all generations (propose better; if you cannot, return the parent). Arm A, run first, added the return-parent clause from generation 2 — disclosed in Appendix A; Arms B/C remove this asymmetry. Arm D likewise uses one fixed prompt for all 50 invocations, with a single disclosed deviation (one extra timeout-warning sentence in seed 789's generation-5 prompt; Appendix A.4). Seeds control harness tie-breaking only; proposer sampling is the dominant, non-seedable stochastic source.

3.4 Precision Sweep Protocol

The precision sweep answers the project's original question — where does the loop break as proposer weight precision drops — and is deliberately a *separate experiment*, not a fifth matched arm: it changes the proposer stack, not just one variable. Proposer: **Qwen2.5-Coder-7B-Instruct** (official Qwen GGUF release), served locally via llama.cpp on 2× NVIDIA T4, at five precisions: FP16 (16.0 effective bits/weight), Q8_0 (8.5), Q4_K_M (4.85), Q3_K_M (3.91), Q2_K (3.35). Per precision: the same 5 seeds × 10 generations (250 loop candidates total) plus 6 zero-shot probes (30 total, the per-precision recall control). Sampling: temperature 0.8, top-p 0.95, a deterministic per-call seed, at most 1,200 new tokens per completion in a 4,096-token context; one chat completion per proposal — this condition is **not agentic** (no tools, no file access), removing the orchestration layer as a confound.

As in Arms A–D, every lineage starts from the same seeded valid parent — the trivial 6×5 grid packing, sum of radii 0.900 — so every lineage has a floor. Protocol differences from Arms A–D, disclosed: (1) the fixed loop prompt matches A.1's constraint text but omits the return-parent clause; (2) the proposer is a different model family and scale, invoked as a bare chat completion rather than an agent, so sweep numbers are compared *within* the sweep (across precisions), never head-to-head against Arms A–D except on the recall question, where zero-shot probes are the common instrument. Integrity discipline is identical: every candidate logged live at evaluation time (`reconstructed:false`), the same deterministic evaluator, plus a SHA-256 hash of each GGUF weight file recorded in the run's provenance log.

Second rung: 14B. The identical protocol was then run with **Qwen2.5-Coder-14B-Instruct** on a four-precision ladder (Q8_0 / Q4_K_M / Q3_K_M / Q2_K; FP16 omitted — the ~29.5 GB file does not

fit 2× T4), adding 224 invocations (200 loop candidates + 24 probes). *Data-integrity disclosure:* the 14B run completed, but the hosting session expired before its output directory was retrieved, losing the per-candidate jsonl, probe raw texts, and checkpoints. What survives — and what every 14B number in this paper is recomputed from — is the run's **verbatim console log** (one line per candidate, printed live at evaluation time, preserved in the notebook's saved version history and archived as `precision_sweep_14b_console.log`). This is a real live log, not a reconstruction, but it lacks fields that were never printed: emitted circle counts, parse-error taxonomy, and raw model text — so the original run's parent-echo analysis could only infer copies from *exact score identity with the lineage's running parent*.

Coordinate-verified re-execution. The rung was therefore re-executed end-to-end (same weights by SHA-256, seeds, prompts, sampling parameters, and generation order) as a platform-executed batch job whose output is saved automatically at completion, with three logging-only additions: per-candidate coordinates in the jsonl, every row echoed redundantly to the console, and quantitative predictions embedded in the runner source *before* execution (the platform's version history timestamps the code ahead of the run; the verbatim prediction block is reproduced in Appendix A.8). We call this a **re-execution**, not a replication: llama.cpp's seeded sampling is deterministic on fixed weights and hardware, so the run regenerates the *same* 224 sampling outcomes rather than drawing fresh ones — and this determinism also means predictions about console-logged quantities (scores, viability, improvement counts) were guaranteed to hold and carry no evidential weight. The only falsifiable predictions were the coordinate-level ones, about information the console log never contained: whether score-matches are verbatim copies or same-sum rearrangements. Both held (Q2_K echo ≥ 60%: observed 94%; each upper rung ≤ 35%: observed 11–16%) — and the check had visible discriminating power: it *refuted* the original score-inferred point estimates on the upper rungs, reclassifying 4 of 12 presumed echoes as rearrangements (Appendix A.8; §5.9). As expected, the re-execution matched the console log bit-for-bit on every field it records — all 224 per-row scores, viability/validity flags, and totals, with the same per-row wall-time pattern — while adding the coordinates (and token counts) the original never recorded. All 14B echo analysis in §5.9 uses this coordinate-logged dataset (`precision_sweep_14b_v2_output/`: 200 candidate rows + 24 probes, `reconstructed:false`); the console log remains the archival record of the original run. A replication in the ordinary sense — fresh seeds, new draws — remains future work.

3.5 Metrics

Per candidate, logged live to `candidates*.jsonl` with `reconstructed:false`: seed, gen, arm, proposer, parent score, score, viable, valid, n_circles, descriptors, parse_error (Arm D: `exec_error` taxonomy), wall clock. Aggregates: best fitness, QD, coverage, viability, validity; Wilson 95% CIs on rates. **Mean best per seed** is defined over *valid* candidates only, excluding the seeded parent; a seed with no valid candidate contributes 0.0 (this is why sweep-table means can fall below the 0.900 seeded floor). Wall-clock was recorded per candidate only for the sweep (LLM inference time) and for Arm D program execution (`exec_wall_s`); Claude-proposer latency and token counts were not logged. Statistical convention: because candidates are nested within lineages (an accepted state persists across generations), **seed-level tests are primary** wherever both levels exist; candidate-level contrasts are reported as descriptive.

4. Experimental Setup

Proposers for Arms A–D are Claude Haiku / Claude Sonnet models invoked as subagents, one invocation per proposal (224 invocations total: 200 loop + 24 probe attempts, of which 23 produced output); the evaluator and archive run locally in Python, no GPU. Both Claude proposers run at full precision. A blunt reproducibility statement: **Arms A–D are not reproducible as run.** The Claude proposers are undated aliases (`claude-haiku` / `claude-sonnet` in the jsonl) invoked through an agent runtime at default, unlogged sampling settings; the qualitative findings replicate across two tiers and a 100-sample control, but the exact runs cannot be re-executed. The precision sweep, by contrast, pins weights (SHA-256) and sampling exactly (§3.4). The sweep runs separately on Kaggle (2× T4, llama.cpp with full GPU offload) and adds 280 invocations at 7B (250 loop candidates + 30 probes, jsonl-logged) plus 224 at 14B (200 loop candidates + 24 probes), run twice — the original (console-log-preserved) and its bit-identical coordinate-logged re-execution (§3.4); with the 100-sample best-of-N control and 6 knowledge probes (§3.3), the study comprises 834 distinct proposer invocations (1,058 total LLM calls issued; the re-execution regenerates the original 224 deterministically rather than sampling new ones).

An earlier exploratory pilot with a different proposer is excluded because its per-candidate data was not logged during the run; only fully-logged runs are reported. All artifacts — harness (`harness.py`, `harness_v2.py`, `harness_v3.py`, `kaggle_precision_sweep.py`, `kaggle_precision_sweep_14b_v2_pushed.py`), per-seed archive states, every proposal file and every proposed program, probe files, and figures — are retained; every number below is recomputed from the logs.

5. Results

5.1 Aggregate Results by Arm

	Arm A (Haiku/unit — pilot)	Arm B (Sonnet/unit)	Arm C (Haiku/rect)	Arm D (Haiku/unit, programs)
Final best per seed	2.5414 ×5	2.541421 ×4, 2.5234 ×1	2.294, 2.167, 2.158, 2.300, 2.167	2.167, 2.479, **2.537**, 1.659, 2.389
Mean ± std (n=5)	2.54142 ± 8×10^{-6}	2.5378 ± 0.0072	2.2171 ± **0.0654**	2.2460 ± **0.3196**
Viability	49/50 (98%), CI [89.5, 99.6]	**50/50 (100%)**, CI [92.9, 100]	**50/50 (100%)**, CI [92.9, 100]	33/50 (66%), CI [52.2, 77.6]
Validity	45/50 (90%), CI [78.6, 95.7]	**50/50 (100%)**	46/50 (92%), CI [81.2, 96.8]	31/50 (62%), CI [48.2, 74.1]
Coverage (of 400 cells)	≤1%	≤0.75%	≤0.75%	≤1.5%
Seeds exceeding ceiling/canonical-analog	0/5	0/5	2/5 climbed mid-run	0/5 reached ceiling; **5/5 improved mid-run**

(Arm A's single non-viable candidate was an orchestration write-failure, not a model format error — at the model level Arm A's format success is 50/50; among all 149 emitted coordinate proposals in Arms A–C, zero failed to parse. Arm D's non-viable candidates fail differently — programs time out or crash rather than mis-format; §5.7.)

Arm A is reported as a pilot, not a co-equal arm. Its gen-2+ prompt disclosed the ~2.54 value and instructed returning the parent when stuck (Appendix A.2) — partially baking in the two properties it exhibits (convergence at 2.5414, zero exceedance). Arm B replicates both properties with a single fixed prompt containing no target value, and the best-of-N control (§5.5) reproduces the ceiling with no loop at all; the finding stands on those, with Arm A as corroborating pilot data.

5.2 Arm A (Pilot): The Loop Never Exceeds the Constructed Value

All five Haiku seeds finish at the 2.5414 packing (25 mutually tangent r=0.1 circles on a 5×5 grid + one gap filler). Seeds differ only early (gen-0 valid scores 0, 2.17, 2.34, 0, 2.17); one seed climbs 2.34 (gen 0) → 2.41 (gen 2, after an invalid gen-1 proposal) → 2.5414 (gen 3); by generation 3 all sit at the ceiling and nothing in the remaining 35 proposals exceeds it. (Pilot caveats above; Arm B is the clean version of this result.)

5.3 Arm B: A Stronger Model, the Same Ceiling

Sonnet, forbidden from running code, converges to the *identical* value — 2.541421 in 4/5 seeds — while showing qualitatively stronger behaviour on the way: hand-derived rigidity arguments for why the canonical packing cannot be locally improved, and small *valid* margin-tightening steps (e.g., 2.5388 → 2.541397 → 2.541419) that asymptotically approach, but never pass, the ceiling. Model tier changes polish — and perhaps reliability, though no reliability difference in our data reaches significance (§5.5) — not the ceiling itself.

The fifth seed is the most instructive datum in the study: its gen-0 proposal was a hex-offset lattice (2.418); elitist selection then locked the lineage into that geometry, and five generations of genuine, verified micro-improvements (2.418→2.501→2.521→2.523) plateaued at a hex-lattice local optimum 2.5234 — *below* the ceiling its four siblings recalled. The loop amplifies whatever the lineage starts from; it does not restructure.

5.4 Arm C: Variance Returns When No Template Fits Cleanly

On the rectangle, no documented solution exists and the square-grid template no longer fits the region natively. Behaviour changes in exactly the way the attractor hypothesis predicts: (i) 4/5 seeds initially emit the *same transferred strategy* (a grid adaptation scoring ≈2.158 — recall of a recipe, not of a solution); (ii) cross-seed variance in final bests is 0.0654 — four orders of magnitude above Arm A; (iii) two seeds show genuine mid-run improvement events (2.167→2.294 at gen 5; 2.167→2.300 at gen 7, after a 2.163 plateau spanning gens 2–5) of a kind never observed on the unit square after the ceiling was reached. When no strong attractor is available, the loop does weak but real search — slow, occasional, and far from any plateau consensus.

5.5 Controls: The Attractor Is Constructed, and the Loop Is Selection

Three controls jointly identify what the loop contributes and where the ceiling comes from.

Original zero-shot probes (n=6 per condition). Haiku/unit: scores 1.49–2.5414, 1/6 at the ceiling value. Sonnet/unit (no-code): 2.418–2.5414, 2/6. Haiku/rectangle: 1.34–2.27 (no canonical exists). These small samples established that both tiers *can* emit the 2.5414 packing zero-shot; their apparent tier difference (1/6 vs 2/6) does not survive larger sampling (below).

Best-of-N control (n=50 per tier, matched invocation budget). Added in revision as the decisive test of the selection hypothesis: 50 independent zero-shot samples per tier — the same 50-invocation budget as one loop arm — each archived verbatim and scored deterministically (protocol §3.3; direct-reply mode, disclosed).

Tier	Viable	Valid	Best of 50	Valid samples at ≈2.5414	Valid samples above 2.5414
Haiku	49/50	45/50	**2.541421**	28/45 (62%)	**0**
Sonnet	50/50	50/50	**2.541421**	29/50 (58%)	**0**

Best-of-50 zero-shot equals the loop's converged value *exactly*, in both tiers. At matched invocation budget, the loop adds nothing beyond selection over the proposer's output distribution — the best-of-N hypothesis is confirmed, not merely asserted. Three revisions to the n=6 picture follow. First, the ceiling-hit rate is ~60% per draw, not 17–33%: the loop's "reliability contribution" is real but modest (best-of-50 delivers the ceiling with near-certainty, loop or no loop). Second, the tier difference dissolves (28/45 vs 29/50). Third, and most important: **0 of 95 valid samples exceed 2.5414** — the attractor caps the raw sampling distribution itself, independent of any loop dynamics.

What the attractor is. Structurally, 94 of the 95 valid control samples are the same object: a 5×5 grid of r≈0.1 circles (or a minor row-shifted variant) plus filler circles — the one exception is a Sonnet sample attempting a central-circle/Apollonian construction that scores far below the grid. Nothing resembling the catalogued ≈2.634 configurations (irregular, unequal radii) ever appears.

Direct knowledge probes (n=3 per tier). Asked point-blank for the best known value and source for this exact objective, **6/6 responses disclaim knowing any published value**; one Sonnet probe correctly recalls that Friedman's *Circles in Squares* catalog covers the objective but cannot recall a number; none names 2.63586, 2.634, or any catalogued value (verbatim responses archived in `probe_knowledge/`).

Together these controls resolve question (1) of §1 in favour of **construction over retrieval**: the models do not know the published record for this objective, never emit its configurations, and instead converge — zero-shot, without any loop — on a template (uniform grid + filler) that any model that understands a grid can construct. The 2.5414 ceiling is a *constructible-attractor* ceiling. We retain the term "recall" only for what Arm C shows transfers across tasks: recall of a *recipe*. What is unambiguous either way: a stronger model does not buy a higher ceiling.

5.6 Coverage and QD

Archive coverage never exceeds 4/400 cells in Arms A–C and 6/400 (1.5%) in Arm D; QD carries no diversity signal and is not interpreted. At these budgets MAP-Elites reduces to elitism, and we report it as such.

5.7 Arm D: Program Evolution Restores Search but Not the Ceiling

Arm D changes one variable against Arm A: the proposer writes a *program* that computes a packing, instead of the packing itself. Same model tier, same task, same seeds, same budget, same evaluator on the program's output.

Three regime changes follow. **First, search returns — on the templated task.** Counting post-first-valid strict improvements uniformly across arms, Arms A/B/C log 5/17/8 events and Arm D logs 12 across all 5 seeds. The raw counts are not the signal — the *destinations* are. Every improving Arm A/B lineage terminates at a single recalled attractor (the canonical 2.5414, or the disclosed hex trap): those climbs are recall completion and margin polish. Arm D's 12 climbs land on five *distinct* non-canonical endpoints spanning 1.659–2.537: seed 456 climbs 2.4385→2.4594→2.4649→2.5340→2.5371 across four accepted mutations of its own champion program — by generation 3 that program is a genuine multi-start optimizer (seven-plus initializations spanning hexagonal, grid, spiral, and corner-biased layouts, a momentum-based position relaxer with radius growth, and perturb-and-reoptimize restarts), written entirely without the proposer ever executing it; seed 1111 climbs four times (2.2653→…→2.3886). Dispersion plus improvement is the signature of search, not recall. The loop is no longer inert, because a program mutation (change a parameter, add a refinement pass) has computable consequences the proposer never has to derive itself.

Second, the ceiling survives — despite a compute subsidy. No seed reaches — let alone exceeds — the 2.5414 attractor ceiling; the best evolved program stops at 2.5371, and the run mean (2.2460) sits *below* Arm A's (2.5414) and even below Arm C's rectangle scores relative to their region. A note on budget currency: "matched budget" here means matched *evaluations* (50 per arm). In *compute*, Arm D is strictly favoured — its programs consumed 471 s of CPU optimization across the run (mean 9.4 s per evaluation, max 30 s) while Arms A–C got zero computation — and it still loses to template construction. Writing a correct constructor-plus-refiner is harder than emitting the template, and 50 evaluations is far too few for evolution to close the gap. This is the scoped answer to whether program-space evolution escapes the attractor: not at this scale. The systems that motivate this study run orders of magnitude more evaluations with richer prompting; our result says the search they perform is real (regime one) but that its advantage cannot be assumed — it must be bought with evaluations.

Third, the failure taxonomy inverts. Arms A–C: 149/149 emitted coordinate proposals parsed; failure was geometric. Arm D: 17/50 candidates are non-viable — 10 timeouts (evolved refinement loops overshooting the 30 s budget), 3 crashes, 3 wrong-count outputs, and 1 program that ran clean and printed 26 circles but with a degenerate (non-positive-radius) entry. Viability, a solved problem for coordinate proposals from capable models, returns as the binding constraint the moment the artifact must *execute*. Cross-seed variance (std 0.320, the highest in the study; same population-std convention

as Arms A–C) is driven by exactly this: seed 789 spent four generations non-viable and finished at 1.659, while seed 1111 — the only seed with zero execution failures (10/10 viable) — tied for the most climb events (4, with seed 456). On a templated benchmark, program evolution reintroduces through the artifact the viability lottery that coordinate proposals had escaped.

Construct integrity: all 50 Arm D proposer invocations were verified to use Read/Write only (48 with exactly two tool calls; two agents used a third call — one cosmetic Edit, one re-Read — both audited post-hoc and confirmed to have executed nothing). The proposer never runs its program; only the harness does.

5.8 Tool-Augmented Agents Break the Ceiling

Six additional Sonnet probes were issued *without* the no-code rule. The agents, unprompted, wrote and ran classical optimizers (SLSQP with analytic gradients, basin hopping, growth-and-repulsion heuristics; 26–88 tool calls per probe). All five probes that produced output are reported, per our own single-run-reporting critique: **2.545298, 2.584992, 2.627064, 2.528356, 2.63598** — every one exceeds the 2.5414 ceiling that no looped condition ever passed; the sixth probe stalled and produced no file. The best, **2.6359805**, is strictly verified by our evaluator at zero tolerance (minimum boundary slack 1.0×10^{-7}, minimum pairwise slack 2.0×10^{-7}) — above AlphaEvolve's published 2.635862 and matching the values (2.63598+) reported by subsequent 2025–26 work in that lineage; we claim a match with the contemporary record, not a new one, and the comparison is a single observation with no error bar on either side. Where 50 loop candidates of pure-LLM proposing never left 2.5414, one agent with a computer effectively solved the task — by delegating search to the right tool. Note that this does not contradict the attractor thesis so much as relocate it one level up: the agent *recalls which solver to write* (SLSQP and basin hopping are textbook choices), and the solver performs the search the LLM cannot. The condition is disclosed as a separate observation (different construct from LLM sampling; provenance and all five scores in `probe_v2/provenance.json`).

5.9 The Precision Sweep: A Capability Floor at 7B, a Coordinate-Verified Novelty Cliff at 14B

The sweep (protocol §3.4) asks the project's founding question: as proposer weight precision drops from FP16 to Q2_K (3.35 effective bits), where does the loop break? The answer is that at 7B scale it was never whole.

	FP16 (16.0 b/w)	Q8_0 (8.5)	Q4_K_M (4.85)	Q3_K_M (3.91)	Q2_K (3.35)
Viability	7/50 (14%), CI [7.0, 26.2]	6/50 (12%), CI [5.6, 23.8]	9/50 (18%), CI [9.8, 30.8]	7/50 (14%), CI [7.0, 26.2]	**16/50 (32%)**, CI [20.8, 45.8]
Validity	7/50	4/50	7/50	6/50	12/50
Mean best per seed ± std	0.985 ± 0.496	0.754 ± 0.414	0.596 ± 0.553	0.982 ± 0.591	1.257 ± 0.186
Best score	1.300	1.163	1.300	**1.800**	1.560
Zero-shot probes valid	0/6	0/6	0/6	0/6	0/6

First, the capability floor dominates. The 7B proposer never emits the 2.5414 packing — or any score near it — at *any* precision: 0/30 zero-shot probes are even geometrically valid, and none of the 250 loop candidates lands at the attractor value. The best score in the entire sweep is 1.800 (seed 1111 at Q3_K_M — exactly double the seeded 0.900 baseline: the lineage discovered that the sparse starting grid's radii could be uniformly doubled, a legitimate but trivial improvement), 29% below the ceiling the Claude arms treat as a floor. The selection-amplifier mechanism (§5.5) explains the inertness: the loop is best-of-N over the proposer's output distribution, and this proposer's distribution contains no high-scoring template to amplify. Where Arm A's viability is 98%, every sweep condition sits between 12% and 32% — the model-scale gap is an order of magnitude; every precision effect within the sweep is a fraction of it.

Second, no cliff. From FP16 down through Q3_K_M — a 4× compression — viability is statistically flat (12–18%, all pairwise Wilson CIs overlapping; FP16 vs Q3_K_M Fisher exact p = 1.0). The hypothesis the project was founded on — a precision threshold below which the loop abruptly degrades — is absent over the entire usable GGUF range at this scale.

Third, the one real effect runs the wrong way, and we can see why. Q2_K viability (32%) is *higher* than FP16's (14%): two-sided Fisher exact p = 0.056 vs FP16 alone, p = 0.007 vs the other four conditions pooled (post-hoc, uncorrected — we flag, not claim, it). The mechanism is visible in the emitted circle *count*. Viability in this sweep is almost entirely count accuracy: across all 250 proposals only 45 contain exactly 26 circles; 153 contain 24–25. FP16, Q8_0, and Q3_K_M have modal count 25 and Q4_K_M ties at 24–25 (14 proposals each) — a systematic undershoot across all four (the models drop 1–2 circles from the 26-circle parent they were shown). Q2_K's count distribution is different in *shape*: broader (tail out to 30–36) and centered on the target (modal count 26, 16/50 proposals). Quantization noise does not improve the model; it broadens a distribution that happened to be biased just below the target, moving mass onto the correct count. One alternative explanation is ruled out directly: the undershoot is *not* token-cap truncation — no 24–25-circle proposal came near the 1,200-token completion cap (maximum 687 tokens; all closed their list bracket and stopped naturally). We call this the **format lottery**: at the capability floor, "viability vs precision" measures the interaction between output-distribution noise and an arbitrary format constraint, not competence. This is also a caution for

quantized-model evaluation generally — near floor performance, added sampling noise can masquerade as improvement on any pass/fail metric with a narrow acceptance region.

Fourth — scaling to 14B lifts the floor and exposes a cliff, in a place viability cannot see. The original 14B run's per-candidate files were lost (its record is the verbatim live console log; §3.4), leaving its echo metric score-inferred. A **coordinate-verified re-execution** (§3.4) — deterministic, so it regenerates the same 224 outcomes bit-for-bit (identical per-row scores; viability totals 22/22/24/19) rather than sampling new ones — upgraded every echo classification below from score-inferred to coordinate-verified; its only falsifiable pre-registered predictions were the coordinate-level ones, and both held (§3.4, A.8). The 14B rung lifts viability far above 7B — 38–48% per rung against 7B's 12–32% on the same quants (12–18% excluding 7B's anomalous Q2_K format-lottery spike), 87/200 vs 38/200 pooled (Fisher $p = 1.7\times10^{-7}$) — while recall stays absent (probes 0/24; best score 1.625, still 36% below the ceiling):

14B	Q8_0 (8.5 b/w)	Q4_K_M (4.85)	Q3_K_M (3.91)	Q2_K (3.35)
Viability	22/50 (44%), CI [31.2, 57.7]	22/50 (44%), CI [31.2, 57.7]	24/50 (48%), CI [34.8, 61.5]	19/50 (38%), CI [25.9, 51.8]
Validity	18/50	20/50	19/50	18/50
Seeds improving past the seeded baseline	**5/5**	**5/5**	**5/5**	**1/5**
Valid outputs echoing their parent (coordinate-verified)	2/18	3/20	3/19	**17/18**

Viability is again flat across precision — all four CIs overlap. But the *content* of the valid outputs changes discontinuously between 3.91 and 3.35 bits. (Echo classification is now **coordinate-verified**: a valid output counts as an echo when its emitted circle set is identical — order-insensitive, at the logs' 6-decimal precision — to its lineage's running parent, the seed's best after the previous generation.) Above the boundary the 14B model behaves like a weak but genuine mutation operator: it edits its parent into slightly better packings, including stepwise micro-improvement chains (one lineage climbs 0.900→0.910→0.915→0.921→0.924→0.926 — five successive accepted improvements), every seed at every precision beats the seeded baseline, and verbatim echoes are rare (8/57 pooled, 14% — the residue of a mutator that sometimes fails to find an edit). At Q2_K the proportions invert into a *copier*: 17 of its 18 valid outputs are coordinate-identical copies of their parent — despite a prompt that demands a strictly higher sum — and only one seed ever improves. The coordinate check also audits the original score-inferred metric both ways: at Q2_K it was exactly right (all 17 score-matches are verbatim copies; zero same-score rearrangements), while above the boundary it *overcounted* — 4 of the 12 score-matches there are different packings that happen to tie their parent's sum (same radius multiset, moved centers). The verified cliff is therefore *sharper* than the score-inferred estimate (14% vs 94%, against the original 21% vs 94%). **The primary test is seed-level**, because candidates are nested within lineages (once a lineage falls into copying, its remaining rows are correlated observations of one event): 15/15 seeds improve above the boundary vs 1/5 below (Fisher $p = 0.001$) — post-hoc, and from a *single* sampling run: the deterministic re-execution regenerates the same draws and adds no statistical evidence (what it adds

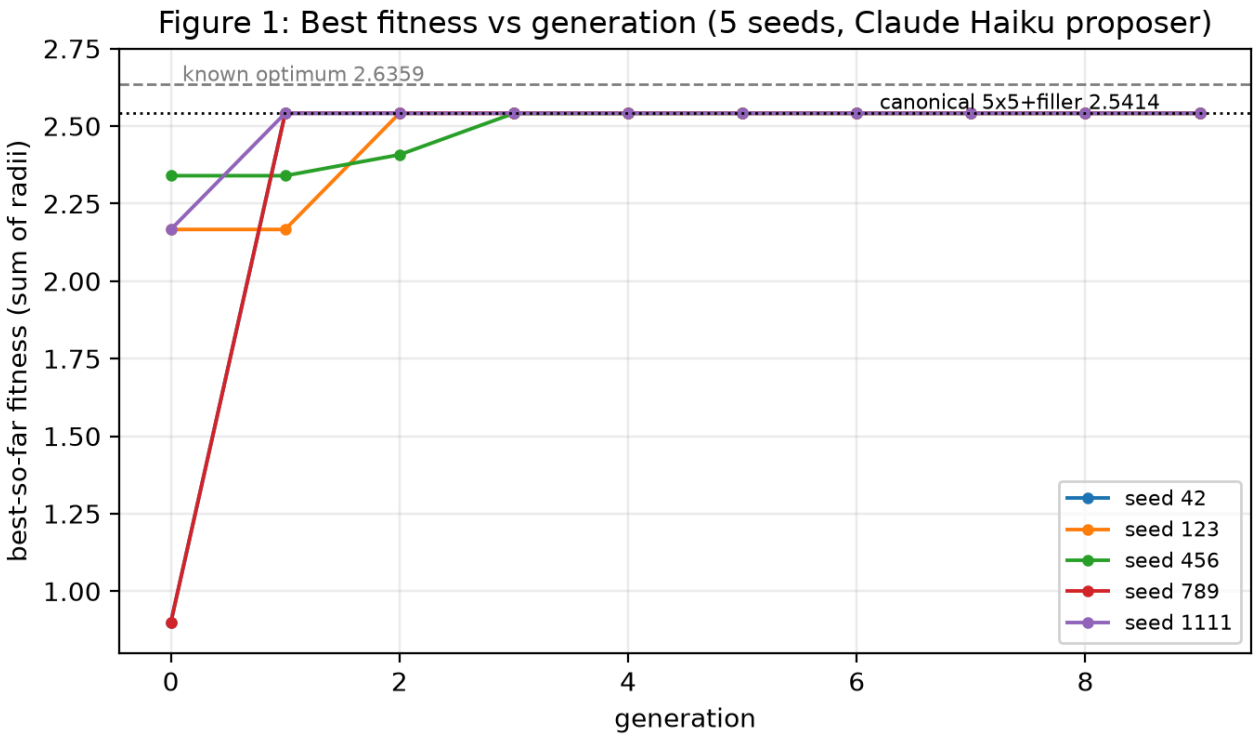
is the coordinate record; §3.4). Fresh-seed replication remains future work. The effect is not driven by any single seed — per-seed echo fractions at Q2_K are 7/7, 4/4, 4/4, 1/1, 1/2, i.e. every seed is majority-echo, versus at most 2 echoes per seed on any upper rung. The candidate-level contrast (17/18 = 94% vs 8/57 = 14%; nominal $p = 5.7\times10^{-10}$) is reported as *descriptive only* — it treats correlated rows as independent and overstates the evidence; the same 5 seeds also recur across rungs, so rungs are not fully independent either. We call this the **novelty cliff**: Q2_K quantization (3.35 effective bits) does not break format-following (viability holds at 38%) or geometric validity (18/50) — it breaks the capacity to *construct a novel valid mutation*, and the model falls back to reproducing the packing already in its context. A viability- or validity-based sweep would report Q2_K as unimpaired. One alternative mechanism remains open: the collapse could be an *instruction-following* failure rather than a constructive one — the model defaulting to echoing its context despite the demand for a strictly higher sum (quantization is known to degrade instruction adherence; Jin et al., 2024). Our zero-shot probes cannot separate the two readings, since every 14B rung fails without a parent in context (0/24); the discriminating probe — supply the parent and demand an output that *differs* from it, valid or not — is Future Work item 1, and until it runs, "novelty cliff" names the behaviour (verbatim copying), not a settled mechanism.

Terminology, used consistently from here on: the **format floor** is the viability threshold (emitting 26 parseable circles at a usable rate); the **recall floor** is the ability to emit the canonical packing at all; **capability floor** denotes sitting below both. 7B sits below both floors; 14B clears the format floor but not the recall floor; the Claude arms clear both.

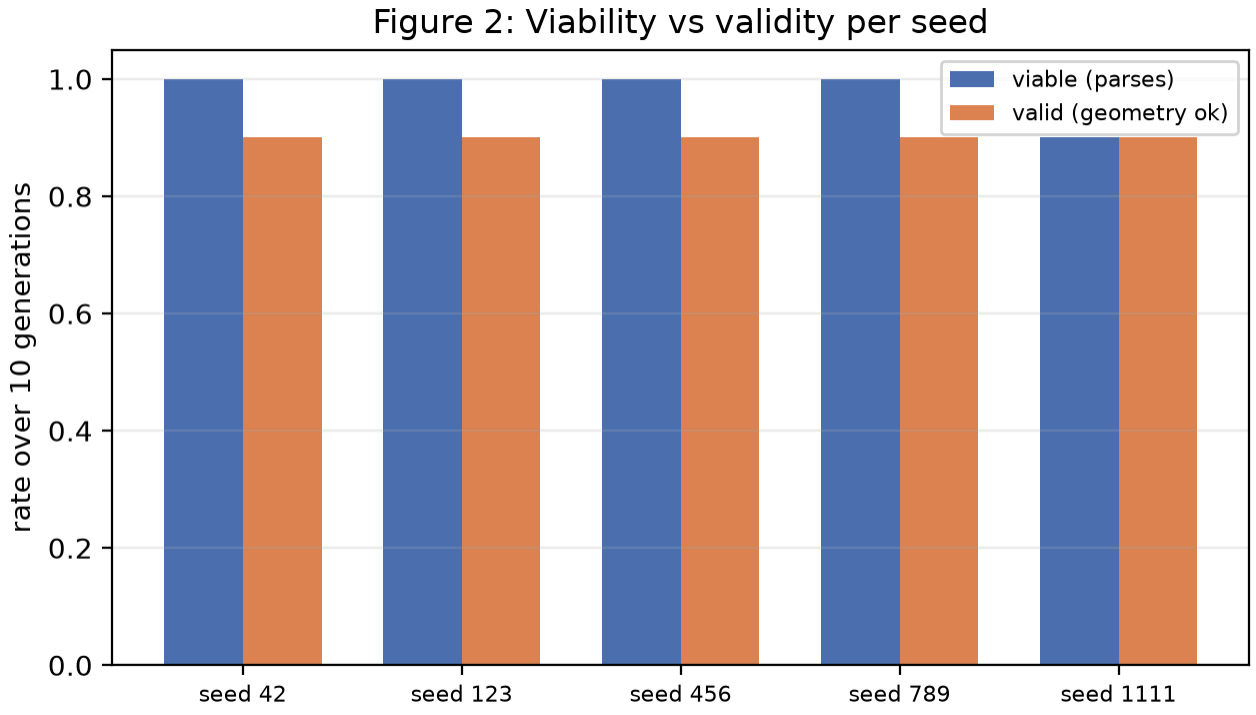
The sweep closes the original question with a two-part answer. At 7B, precision is second-order to scale: below the capability floor there is no recall for quantization to erode and no format reliability for it to break — both were already gone at FP16. At 14B, above the format floor, a real precision cliff finally appears — but in proposal *novelty*, not in the pass/fail axes the cliff hypothesis anticipated, and recall remains untouched at every precision because it never existed at this scale. The original question — does quantization lower the *recall ceiling* before it breaks format-following — still requires a proposer with recall to lose (a model whose full-precision behaviour matches the Claude arms; §Future Work).

5.10 Figures

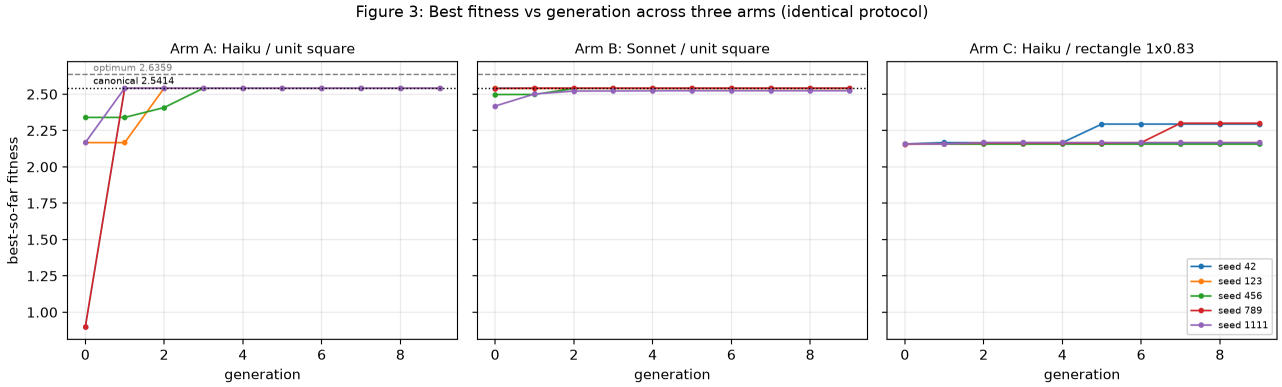
- **Figure 1** (`agent-run/figures/fig1_fitness_vs_gen.png`): Arm A best-so-far vs generation (seed 42 occludes 789).



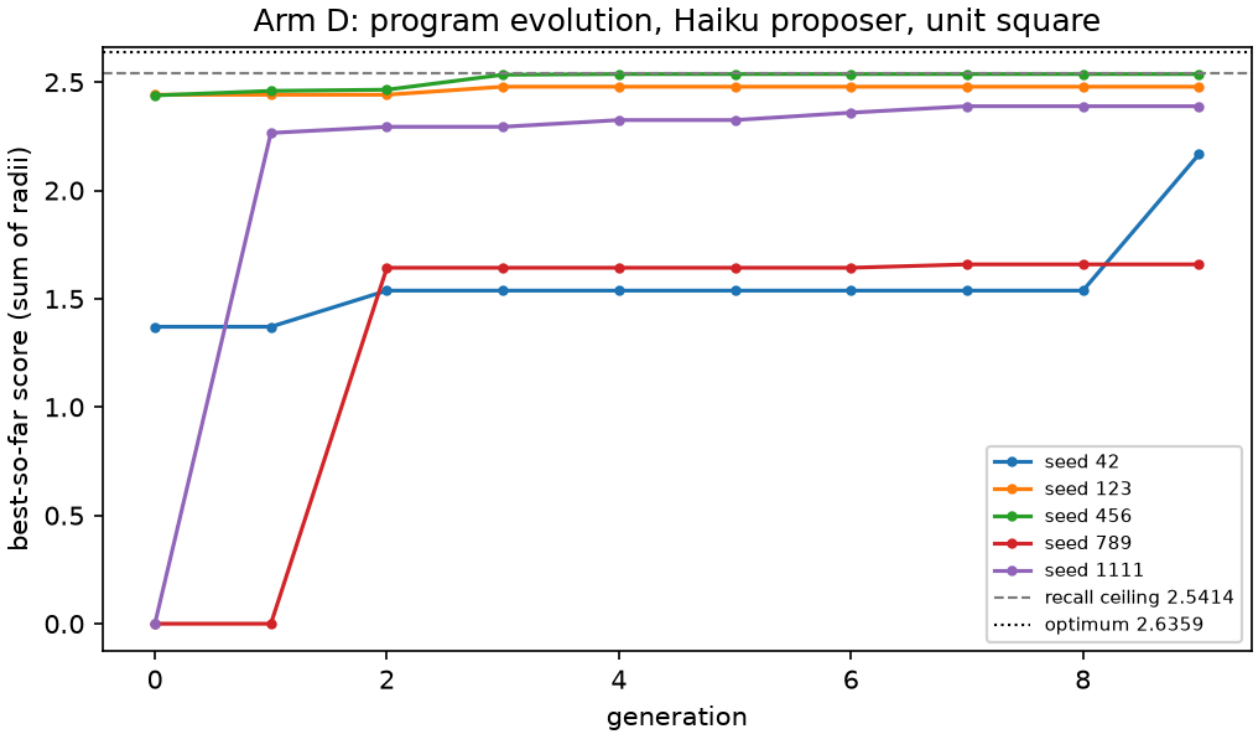
- **Figure 2** (`agent-run/figures/fig2_viability_validity.png`): Arm A viability vs validity per seed.



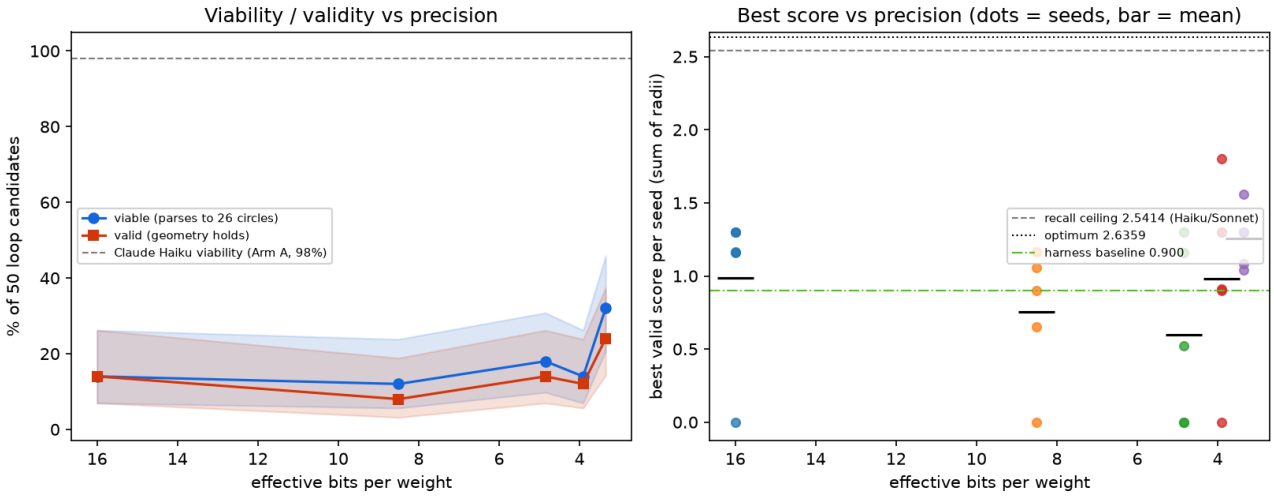
- **Figure 3** (`agent-run/figures/fig3_three_arms.png`): Arms A–C, matched protocol (one disclosed prompt asymmetry in Arm A; §3.3, A.2) — the plateau consensus in A/B vs the dispersed, still-moving trajectories in C is the central picture for the memorability axis.



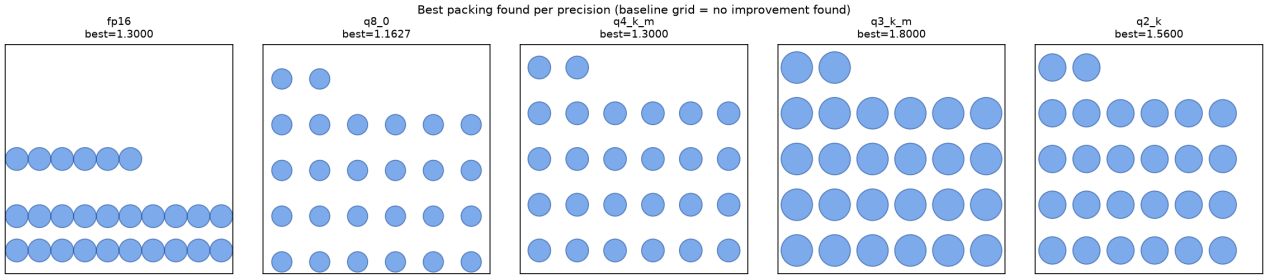
- **Figure 4** (`agent-run/figures/fig4_program_evolution.png`): Arm D best-so-far per seed against the attractor ceiling (2.5414) and the best known value (2.6359) — five dispersed, still-climbing trajectories, none reaching the ceiling.



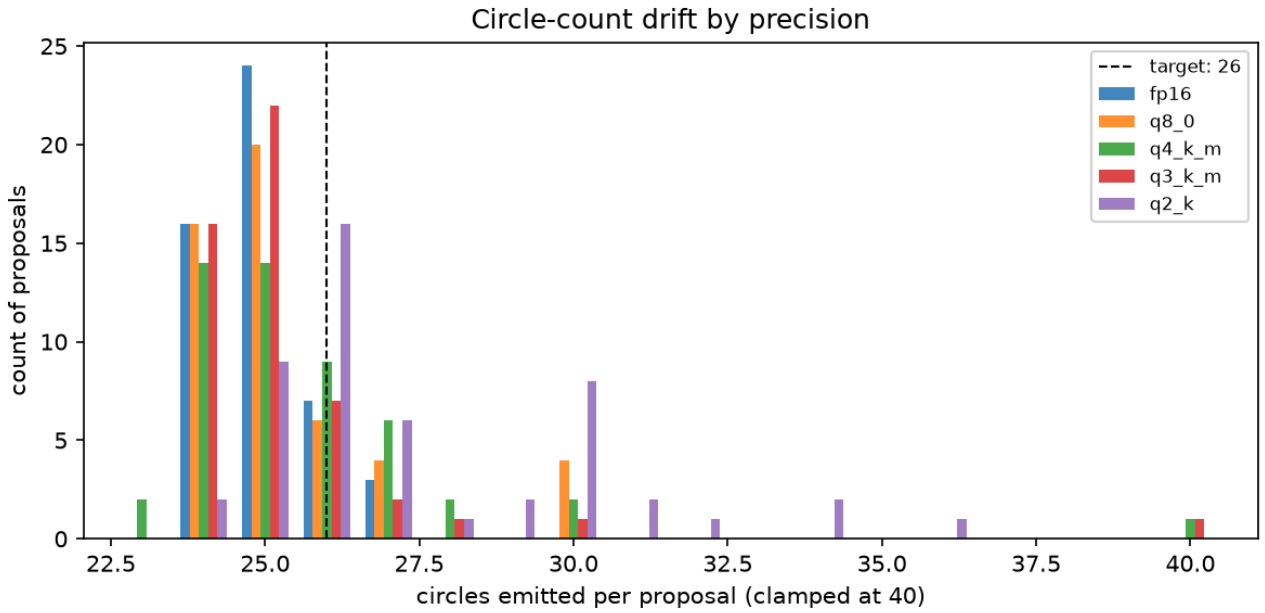
- **Figure 5** (`agent-run/figures/fig5_precision_cliff.png`): the sweep's central picture — viability/validity vs effective bits (flat, with the 2-bit uptick, all far below the Haiku 98% reference line) and best valid score per seed vs the recall ceiling.



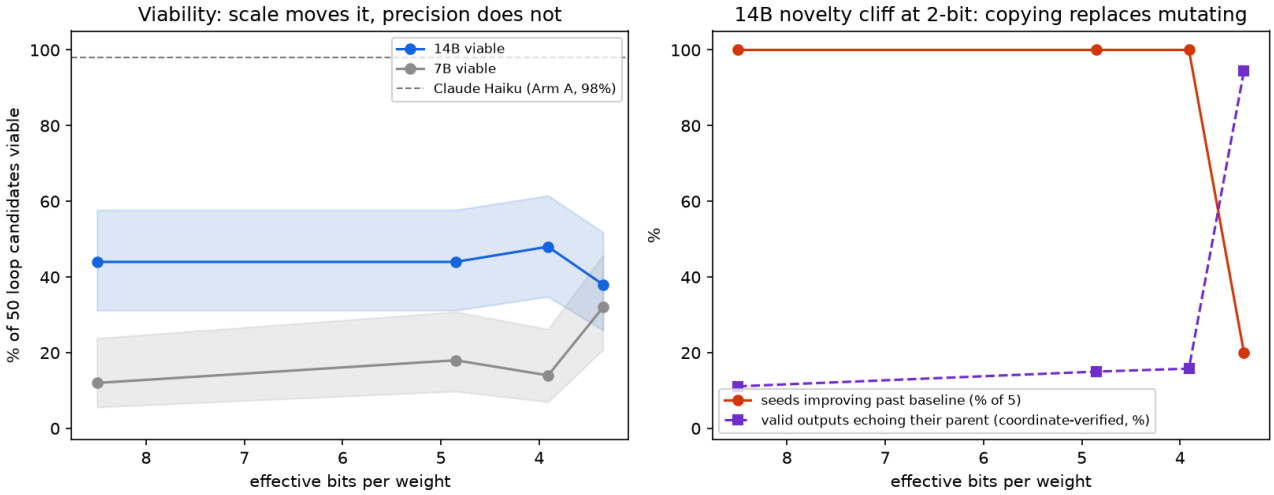
- **Figure 6** (`agent-run/figures/fig6_packing_portraits.png`): best packing found at each precision, drawn to scale — every panel is a sparse grid variant; nothing resembles the canonical dense packing.



- **Figure 7** (`agent-run/figures/fig7_count_drift.png`): circle-count histograms per precision — the format-lottery mechanism: modal undershoot at 24–25 for FP16→Q3_K_M, Q2_K's broader distribution centered on 26.



- **Figure 8** (`agent-run/figures/fig8_14b_novelty_cliff.png`): the 14B rung — left, viability vs bits for 7B and 14B (scale moves the level, precision does not); right, the novelty cliff at Q2_K: seeds-improving collapses (100%→20%) exactly where the coordinate-verified parent-echo rate spikes (14%→94%).



6. Discussion

6.1 Selection Amplifier over a Constructible Attractor, Not Search Engine

Across 100 unit-square coordinate candidates from two model tiers, nothing ever exceeded the packing both models emit zero-shot roughly 60% of the time — and the best-of-50 control (§5.5) shows why: at

matched invocation budget, pure selection over the zero-shot distribution reaches the loop's converged value exactly, and 0/95 valid raw samples exceed it. The loop's contribution is best-of-N selection, empirically confirmed, not discovery. The mechanism, per the knowledge probes, is construction rather than retrieval: the models cannot cite the published record for this objective and never emit its configurations; they converge on a self-constructible template. This reading applies to *coordinate* evolution specifically: Arm D's program lineages are not attractor-bound (they search; §5.7) but are budget-starved, and they too never exceeded the attractor value. Single-run SOTA claims on benchmarks with strong constructible attractors are therefore ambiguous between search and reproduction — with one clean exception: a result that *exceeds* the published record (AlphaEvolve's 2.63586 here) cannot be reproduction of anything and is not impugned by our data. Our claim targets the unexamined middle: when a reported value sits at or below what the model can produce without the loop, the default reading should be reproduction, and a best-of-N control is the cheap test.

6.2 The Ceiling Is the Artifact's, Not the Model's

The sharpest evidence is the tier comparison: Haiku and Sonnet share the 2.5414 ceiling to six decimal places. If the ceiling were a capability limit we would expect it to move with model strength; it does not. It moves only when the *task* leaves the training distribution (Arm C) or when the agent is allowed to *compute at proposal time* rather than recall (§5.8) — and notably it does **not** move when computation is merely embedded in the evolved artifact at matched budget (Arm D, §5.7).

6.3 What Elitism Does to an Attractor-Bound Loop

Two failure modes bracket the loop's behaviour: four Arm B seeds show attractor lock-in *at* the ceiling; the fifth shows lineage lock-in *below* it — five generations of real, verified improvement inside a basin the ceiling packing does not occupy. Elitist LLM-guided loops amplify their gen-0 draw, whatever it is. Restart or diversity mechanisms, not more generations, are the lever.

6.4 Variance Is a Benchmark Property

The variance ladder — 8×10^{-6} (templated square) → 0.0072 (one trapped lineage) → 0.0654 (template-hostile rectangle) — plus the earlier observation that low-viability proposers manufacture variance by wasting generations, implies that cross-seed variance in this literature is governed by benchmark memorability and proposer viability. Multi-seed reporting remains necessary, but *what the variance means* differs by regime: on memorizable tasks it indicts viability; on novel tasks it reflects genuine search stochasticity.

6.5 The Compute-Access Spectrum

The four conditions order themselves by *where computation is allowed to live*: nowhere (Arms A–C, coordinate lists — the proposer must know or derive every digit), inside the evolved artifact (Arm D — the program computes what the proposer cannot), or at proposal time (§5.8 — the agent runs an optimizer before answering). The results ladder accordingly: recall-bound convergence at 2.5414 → genuine but

budget-starved search peaking at 2.5371 → 2.63598 in one invocation. Two lessons. First, the oft-cited advantage of program-space evolution is real but not free: at 50 evaluations it *loses* to template construction on a templated task, so headline results from program-evolution systems should be read jointly with their evaluation budgets. Second, the spectrum's top rung is not an evolutionary loop at all — direct tool orchestration dominated every looped condition at a fraction of the invocations.

6.6 Precision Is Second-Order to Scale

The sweep adds a fourth axis to the study's variable set — weight precision — and finds it second-order to every other knob until the model clears the format floor, at which point its one real effect appears somewhere pass/fail metrics do not look. At 7B, changing precision from 16 to 3.35 effective bits changed *almost nothing*, because the proposer was already below the capability floor at full precision. At 14B, precision left viability, validity, and (absent) recall untouched — and then broke exactly one thing, catastrophically: the capacity to propose a *novel* valid mutation, which collapsed at 2-bit into verbatim parent-copying (coordinate-verified; §5.9). The practical ordering for anyone building these loops, consistent with §6.5's spectrum: model scale ≫ tool access > artifact type > attractor availability (for what variance means) ≫ weight precision — until Q2_K (3.35 effective bits), where precision suddenly binds the one property an evolutionary loop cannot function without. Two corollaries for the quantization literature. First, pass/fail metrics evaluated near a model's floor are unreliable instruments for precision effects — our 7B 2-bit condition "outperforms" FP16 through pure output-noise interaction with the acceptance region. Second, pass/fail metrics evaluated *above* the floor can be equally misleading in the opposite direction: our 14B 2-bit condition looks unimpaired on viability and validity while having lost the ability to do the task's actual job. Precision evaluations need behavioural metrics (novelty, echo rate, improvement rate), not just acceptance rates.

6.7 Implications

- Evaluate LLM search on tasks with neither published optima nor strong constructible templates; report a zero-shot control — ideally a best-of-N control at matched invocation budget — alongside any loop result. A best-of-N control is cheap and decisive: if it matches the loop, the loop is selection.
- Report viability and validity separately; near-perfect format-following (149/149 emitted coordinate proposals parsed) means geometry, not formatting, is the frontier for capable proposers — until the artifact must execute (Arm D), where viability re-becomes the frontier.
- If the goal is solution quality rather than studying the loop, give the agent tools: LLM-orchestrated classical optimization dominated every pure-LLM condition by a wide margin at a fraction of the invocations.
- When evolving programs, report the execution-failure taxonomy (timeout/crash/parse), not a single viability rate — in our data it is the binding constraint and the main variance driver.
- Do not measure precision effects with pass/fail metrics on models near their capability floor: acceptance-region noise (our format lottery) can invert the expected ordering. Establish that the FP16 model clears the task before sweeping its quantizations.

7. Conclusion

In a fully auditable study, LLM-guided evolutionary search never out-searched a value its proposer could produce without the loop. Both a small and a mid-tier model converge to the same 2.5414 grid-and-filler packing they each emit zero-shot roughly 60% of the time; best-of-50 zero-shot sampling reaches exactly the loop's converged value at matched invocation budget in both tiers (0/95 valid samples above it); and direct knowledge probes show the models do not know the published record for this objective — the ceiling is a *constructible attractor*, not retrieval, and the loop is a selection amplifier over it. A stronger tier does not buy a higher ceiling, and one lineage shows the loop can trap a model below its own attractor. On a rectangle variant with no natural template, seed variance re-emerges and the loop performs weak genuine search. Switching the evolved artifact from coordinates to programs restores genuine search — all five seeds climb — yet at matched evaluation budget, and despite a compute subsidy Arms A–C never received, the best evolved program still falls short of the attractor while viability collapses to 66%. Agents permitted to run code at proposal time escape the ceiling entirely: all five tool probes beat it, the best matching the contemporary published record in a single invocation. The precision sweep answers the project's founding question with a twist: at 7B there is no cliff because the model sits below a capability floor with no recall to erode and no format reliability to break; at 14B — above the format floor — a cliff appears, between 3.91 and 3.35 effective bits, in proposal *novelty*: seeds improving collapse from 15/15 to 1/5 ($p = 0.001$, seed-level primary test) as valid outputs shift from 14% to 94% verbatim parent-copies — coordinate-verified by a bit-identical re-execution of the run (statistics rest on that single sampling run; fresh-seed replication is future work). Quantization's first casualty in this loop is not correctness but the capacity to propose something new — invisible to every pass/fail metric in the sweep. The constructive reading: the LLM-as-mutation-operator paradigm, evaluated on benchmarks with strong constructible attractors, measures selection over reproduction; a cheap best-of-N control can diagnose this in any such system; program-space evolution performs real search whose advantage must be bought with evaluations; achieving *results* favours LLM-orchestrated tools; and precision experiments need proposers above the floor. All 774 jsonl-logged candidates and control samples (200 Claude loop + 250 Qwen-7B loop + 100 best-of-N + 224 from the 14B coordinate-logged re-execution) carry `reconstructed:false`; the original 14B run — whose per-candidate files were lost to session expiry, disclosed in §3.4 and Limitations — is documented by a verbatim live console log that the re-execution reproduces bit-for-bit; all retained probe outputs and proposed programs are verbatim, and the sweep's weight files are SHA-256 pinned. All artifacts are retained by the author and available on request; a public archive is in preparation.

Limitations

- **The precision sweep tests one model family at two scales.** Qwen2.5-Coder 7B and 14B; both sit below the *recall* floor, so the founding question (does quantization lower the recall ceiling?) remains untested — our findings are scoped to the format and novelty axes. Effective bits-per-weight values are llama.cpp nominal figures, and precision co-varies with the quantization algorithm (K-quant mixed schemes), not bit width alone. The 7B Q2_K viability inversion is post-hoc and uncorrected for multiple comparisons; the 14B novelty cliff is post-hoc and rests on a

single sampling run — the deterministic re-execution regenerates the same outcomes rather than drawing fresh ones, so it verifies the record, not the statistics (§3.4); replication on new seeds remains future work. The sweep also differs from Arms A–D in harness (non-agentic single-turn completion, seeded gen-0 parent, no return-parent clause), so only within-sweep comparisons and the zero-shot recall control are interpreted across experiments.

- **The original 14B run's per-candidate files are lost; a bit-identical re-execution is the analysis record.** The session hosting the completed original run expired before download, destroying its jsonl, probe raws, and checkpoints (§3.4); its verbatim live console log is archived. The coordinate-logged re-execution (§3.4) reproduced that log bit-for-bit and supplies the coordinates, so the echo analysis no longer rests on score inference — and the score-identity concern this bullet originally raised proved real in the direction we could not previously check: 4 of the 12 upper-rung score-matches are different packings tying their parent's sum (misclassified as echoes by the old metric), while all 17 Q2_K score-matches are verbatim copies. Residual caveat: the re-execution's bit-identity to the original run is verified against the console log's printed fields (scores, viability flags, wall-time pattern); fields the original never printed are attested by the re-execution alone.
- **Scale.** n=5 seeds × 10 generations per arm; the variance ladder is directionally strong (4 orders of magnitude) but its levels are point estimates. CIs on rates are reported; cross-arm significance testing at n=5 is intentionally avoided.
- **The rectangle manipulation is bundled.** Moving from the unit square to 1×0.83 changes the region, the attainable score scale, and how naturally grid templates fit — simultaneously with documentation status. The variance-ladder contrast therefore has multiple candidate causes; the cleaner manipulation (same region, undocumented parameters — e.g., N=23 or N=27 in the unit square, which the AlphaEvolve lineage never touched) is future work. Rectangle non-documentation is itself an assumption: no published solution was found for 1×0.83/N=26, but absence from our search is weaker than proof of absence from training data.
- **Pseudo-replication risk in candidate-level statistics.** Loop candidates are nested within lineages; accepted states persist. All candidate-level Fisher contrasts are therefore descriptive; seed-level tests are primary (§3.5, §5.9). The same 5 seeds recur across precision rungs, so rungs are not fully independent samples either.
- **The best-of-N control differs from the original probes in invocation mode** (direct text reply vs file-writing agents), one day later under the same undated aliases; its large canonical-rate difference from the n=6 probes (≈60% vs 17–33%) is most plausibly sampling noise at n=6, but a mode contribution cannot be excluded.
- **Agentic proposers.** Proposals come from agents with file access (read state, write proposal); Arm A's viability figure counts one orchestration write-failure. The no-code rule was enforced by instruction and verified by tool-use counts (all B/C agents: exactly read+write; Arm D: 48/50 exactly read+write, 2 audited exceptions with no execution), not by sandboxing. Arm A's jsonl predates the per-row `proposer` field added in later harness versions, so its Haiku attribution rests on the run configuration rather than a per-row record; sampling settings for all Claude proposers are the agent runtime's defaults and are not logged (§4).
- **Elitist selection only;** archive coverage ≤1.5% makes the MAP-Elites framing nominal.

- **Program evolution tested only at matched (small) budget.** Arm D evolves programs like the motivating systems do, but at 50 evaluations with a single lineage per seed, stdlib-only imports, and a 30 s runtime cap — AlphaEvolve-class systems use richer prompting, populations, and orders of magnitude more evaluations. Our finding is that program evolution does not beat recall *at this budget*; whether it escapes the ceiling at scale remains open, and our data (real climb events, budget-limited) is consistent with either outcome.
- **The Arm D sandbox is best-effort, not a security boundary.** Imports and dangerous names are rejected by a static AST allowlist plus `python -I` isolation and a 30 s kill; a determined program could in principle evade a static filter (e.g., string-constructed attribute access). All 50 proposed programs are retained verbatim and none attempts evasion, but the stdlib-only guarantee is enforced by inspection, not by a hardened runtime.
- **One benchmark family.** Circle packing throughout; the constructible-attractor claim should be tested on other templated/non-templated task pairs.

Future Work

1. **Probe the mechanism of the novelty cliff:** at Q2_K, ask the model for *any* valid packing with no parent in context (zero-shot probes already show 0/6) versus a packing *required to differ* from a provided parent — separating "cannot construct novelty" from "defaults to copying despite instruction." Also: fresh seeds and other 2-bit quantizations (IQ2, alternative K-quant mixes) to test whether the cliff is a property of the bit width or of the Q2_K scheme.
2. **The precision sweep above the attractor floor:** repeat the ladder with proposers that emit the 2.5414 template at full precision (32B+ locally, or serving-level contrasts such as bf16-vs-fp8 of a 70B via quantization-pinned API routing) — the original question, "does quantization lower the attainable ceiling before it breaks format-following?", is only answerable where a ceiling exists to lower.
3. **Same-region undocumented variants:** N=23 or N=27 in the unit square, to unbundle the rectangle manipulation (§Limitations) — documentation status changes while region, scale, and template fit stay fixed.
4. **Templated/novel task pairs** in other domains (bin packing with custom item sets, graph layouts) to generalize the variance-ladder result.
5. **Anti-lock-in mechanisms:** restarts, multi-island archives (FunSearch-style; Romera-Paredes et al., 2024), and non-elitist selection measured specifically by exceed-the-attractor rate.
6. **The tool-augmented frontier:** systematic comparison of LLM-orchestrated optimizers vs pure-LLM loops at matched wall-clock and cost.
7. **Program evolution at scale:** extend Arm D along the evaluation-budget axis (500, 5000 evaluations; populations instead of single lineages) to locate the budget at which program-space search overtakes the attractor ceiling — the crossover point our matched-budget result predicts must exist if the motivating systems' claims hold.

References

(All entries web-verified against primary sources, 2026-07-17/18.)

1. Romera-Paredes, B., Barekatain, M., Novikov, A., et al. (2024). *Mathematical discoveries from program search with large language models*. Nature 625, 468–475. DOI: 10.1038/s41586-023-06924-6.
2. Novikov, A., et al. (2025). *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. arXiv:2506.13131.
3. Sharma, A. (2025). *OpenEvolve: an open-source implementation of AlphaEvolve*. GitHub: codelion/openevolve.
4. Lange, R. T., Imajuku, Y., & Cetin, E. (2025). *ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution*. arXiv:2509.19349 (ICLR 2026).
5. Huang, Z., Wu, W., Wu, K., Wang, J., & Lee, W.-B. (2025). *CALM: Co-evolution of Algorithms and Language Model for Automatic Heuristic Design*. arXiv:2505.12285.
6. Mouret, J.-B., & Clune, J. (2015). *Illuminating search spaces by mapping elites*. arXiv:1504.04909.
7. Meyerson, E., Nelson, M. J., Bradley, H., et al. (2023). *Language Model Crossover: Variation through Few-Shot Prompting*. arXiv:2302.12170; ACM Trans. Evol. Learn. Optim.
8. Lehman, J., Gordon, J., Jain, S., et al. (2022). *Evolution through Large Models*. arXiv:2206.08896.
9. Guo, Q., Wang, R., Guo, J., et al. (2024). *EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers*. ICLR 2024. arXiv:2309.08532.
10. Nasir, M. U., Earle, S., Cleghorn, C. W., et al. (2024). *LLMatic: Neural Architecture Search via Large Language Models and Quality Diversity Optimization*. GECCO 2024. arXiv:2306.01102.
11. Dettmers, T., & Zettlemoyer, L. (2023). *The Case for 4-bit Precision: k-bit Inference Scaling Laws*. ICML 2023, PMLR 202:7750–7774. arXiv:2212.09720.
12. Jin, R., Du, J., Huang, W., et al. (2024). *A Comprehensive Evaluation of Quantization Strategies for Large Language Models*. ACL Findings 2024. arXiv:2402.16775.
13. Li, Z., et al. (2025). *Quantization Meets Reasoning: Exploring LLM Low-Bit Quantization Degradation for Mathematical Reasoning*. arXiv:2501.03035.
14. Friedman, E. (maintained 2011–present). *Circles in Squares* (best known packings maximizing the sum of radii in a unit square). erich-friedman.github.io/packing/cirRsqu/. N=26 records: Cantrell (2011), Specht (2012), ≈2.634.
15. Sainz, O., Campos, J. A., García-Ferrero, I., et al. (2023). *NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark*. Findings of EMNLP 2023. arXiv:2310.18018.
16. Golchin, S., & Surdeanu, M. (2024). *Time Travel in LLMs: Tracing Data Contamination in Large Language Models*. ICLR 2024. arXiv:2308.08493.
17. Deng, C., Zhao, Y., Heng, Y., et al. (2024). *Unveiling the Spectrum of Data Contamination in Language Models: A Survey from Detection to Remediation*. Findings of ACL 2024. arXiv:2406.14644.

18. Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., & Mirhoseini, A. (2024). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*. arXiv:2407.21787.

Appendix A: Prompts (verbatim)

A.1 Loop proposer prompt (Arms B, C — one prompt for all generations). The agent may use only the Read and Write tools ("do NOT run Bash/PowerShell or any code; the packing must come from your own reasoning"), reads the seed's state file, and:

> Produce a packing with a HIGHER sum of radii than best_score if you can; if you cannot find a strictly better valid packing, return the parent unchanged. MUST hold: exactly 26 circles; all inside the [region]; no overlaps (center distance ≥ sum of the two radii). Write ONLY the raw Python list of 26 [x,y,r] lists.

A.2 Arm A prompt evolution (disclosed). Arm A, run first, used the same structure but added "If best_score ~2.54 you likely cannot beat it — then just return the parent unchanged" from generation 2 onward (generations 0–1 lacked it). This retention bias motivated the single fixed prompt used in Arms B and C.

A.3 Zero-shot probe prompt. Same task statement, no parent, no loop; no-code variants add the strict tool rule; the unconstrained variant (§5.8) omits it.

A.4 Arm D proposer prompt (one fixed prompt, all 50 invocations). The agent reads the seed's context file (current best score + full source of the champion program, fed back verbatim) and writes a program file. Core text:

> The program's task: compute a packing of exactly 26 non-overlapping circles inside the unit square [0,1]x[0,1], maximizing the sum of radii (that sum is the score). It may import ONLY: math, random, itertools, functools. No other imports (no numpy, no scipy — it will be rejected). It must finish in under 25 seconds and print exactly one line to stdout: a Python list of 26 lists [x, y, r] of floats. [...] Invalid geometry scores zero, so include a repair or shrink step [...]. The program itself CAN do real computation: constructive placement plus iterative refinement (grow radii, relax positions, local search). That is encouraged — put the optimization inside the program. If the context file shows a champion program, treat it as the parent: mutate or improve it [...], or replace it entirely if you have a better idea. Hard rules for YOU: Use ONLY the Read and Write tools. Do NOT run any code, do NOT use Bash, do NOT test the program.

Disclosed deviation: seed 789's generation-5 prompt contained one extra sentence warning that the previous program had timed out and advising a capped iteration count; all other 49 invocations used the identical template. *Tool-use audit:* 48/50 agents made exactly two tool calls (Read context, Write program); the two three-call agents (seed 1111, gens 4 and 7) were resumed post-run and asked to

enumerate their calls — one made a cosmetic Edit (removing an unused import), one re-Read the context file; neither executed anything.

A.5 Precision-sweep prompts (one fixed loop prompt, all 250 invocations; one fixed probe prompt, all 30). Non-agentic: a single chat completion, no tools. Loop prompt (verbatim, with the parent packing serialized in place):

> You are the proposal step of an evolutionary search for circle packings. Current best packing (sum of radii = {best_score}): {packing}
>
> Produce a packing of exactly 26 circles with a HIGHER sum of radii. MUST hold: exactly 26 circles; every circle fully inside the unit square [0,1]x[0,1] (x-r >= 0, x+r <= 1, same for y); no two circles overlap (distance between centers >= sum of the two radii; touching is allowed). Output ONLY the raw Python list of 26 [x, y, r] lists. No explanation, no code fences, no other text.

Zero-shot probe prompt: the same task statement without the parent packing ("Pack exactly 26 non-overlapping circles inside the unit square [0,1]x[0,1] so that the sum of their radii is as large as possible..."), same output-format instruction. Note the loop prompt intentionally omits A.1's return-parent clause; parsing strips code fences and extracts the outermost bracketed list before `ast.literal_eval`, so fenced-but-otherwise-correct outputs are not counted as failures.

A.6 Best-of-50 control and knowledge-probe prompts (one fixed prompt each; all 100 + 6 invocations). Best-of-50 (verbatim):

> Pack exactly 26 non-overlapping circles inside the unit square [0,1]x[0,1] so that the sum of their radii is as large as possible. MUST hold: exactly 26 circles; every circle fully inside the unit square (x-r >= 0, x+r <= 1, same for y); no two circles overlap (distance between centers >= sum of the two radii; touching is allowed). Do not use any tools and do not run any code — the packing must come from your own reasoning. Your entire reply must be ONLY the raw Python list of 26 [x, y, r] lists. No explanation, no code fences, no other text.

Knowledge probe (verbatim):

> Answer from your own knowledge only; do not use any tools. Question: for the problem of packing exactly 26 non-overlapping circles (radii may differ) inside the unit square so as to MAXIMIZE THE SUM OF THE RADII, what is the best known value and configuration? If you know a specific published value, state the number and its source. If you do not know any published result for this exact objective, say so plainly. Reply in at most 120 words.

All 106 raw responses archived verbatim (`probe50/`, `probe_knowledge/`); scoring by `score_probe50.py` (same evaluator rules, EPS 10^{-6}); one jsonl row per response, appended at evaluation time.

A.7 Harness version map. `harness.py` = Arm A (adds live jsonl logging, MAP-Elites archive, evaluator); `harness_v2.py` = Arms B/C (adds per-row `arm`/`proposer` fields, rectangle region support, fixed single prompt); `harness_v3.py` = Arm D (adds AST-allowlist sandbox, `python -I` execution with 30 s kill, `exec_error` taxonomy, program-source parent feedback); `kaggle_precision_sweep.py` = 7B and original 14B sweeps (llama.cpp chat-completion proposer, checkpoint/resume, SHA-256 weight pinning, per-call seeds; 14B run differs only in `REPO`, ladder, and paths); `kaggle_precision_sweep_14b_v2_pushed.py` = the 14B re-execution runner — identical protocol, plus per-candidate coordinate logging, redundant console echo of every jsonl row, per-condition zip snapshots, and the registered-predictions header (A.8). Evaluator geometry rules (26 circles, in-bounds, pairwise non-overlap, EPS 10^{-6}) are identical across all versions.

A.8 The re-execution's registered predictions, verbatim, with an honest audit of which could fail. The following block is reproduced exactly from the header of `kaggle_precision_sweep_14b_v2_pushed.py`, the runner pushed to the execution platform before the re-execution ran (the platform's version history timestamps the code ahead of execution):

> PREREGISTERED PREDICTIONS — written and pushed BEFORE this run executed. [...] Predictions for THIS coordinate-logged replication (rate-level, not row-level — llama.cpp sampling is not bit-reproducible across builds): **P1** Improvement: seeds improving past the seeded baseline will be >=4/5 at each of q8_0, q4_k_m, q3_k_m, and <=2/5 at q2_k. (Original: 5/5, 5/5, 5/5 vs 1/5.) **P2** Parent-echo (now COORDINATE-VERIFIED: proposal circles == lineage's running parent circles): among valid rows, q2_k echo rate >= 60%; each upper rung <= 35%. (Original, score-inferred: 17/18 = 94% vs 4/18, 4/20, 4/19 = 20-22%.) **P3** Viability flat across rungs (95% CIs overlap; no monotone cliff). (Original: 22, 22, 24, 19 of 50.) **P4** Zero-shot probes ~0 valid (original 0/24); canonical 2.541421 grid never emitted anywhere (original 0/224). Disconfirmation: P1 AND P2 both failing at q2_k falsifies the novelty cliff; the paper section would be withdrawn, not softened.

Audit. The predictions were written expecting statistical replication ("rate-level, not row-level"); in the event, the same wheel version and hardware made sampling fully deterministic, which changes their evidential status and we report it plainly: **P1, P3, and P4 concern quantities recoverable from the archived console log and were therefore guaranteed to hold once determinism was observed — they carry no evidential weight.** The only falsifiable content was **P2's coordinate-level claims**, about information no surviving record contained: whether score-matches are verbatim copies. Both P2 bounds held (Q2_K observed 94% ≥ 60%; upper rungs observed 11–16% ≤ 35%), and the coordinate check demonstrated discriminating power by refuting the original score-inferred upper-rung point estimates (8 true echoes, not 12 — four were same-sum rearrangements; §5.9). The disconfirmation rule was live for P2: had Q2_K's score-matches proven to be rearrangements rather than copies, the cliff section would have been withdrawn.